NTNU | Norwegian University of Science and Technology

Data Privacy and Privacy Preserving Techniques

Dr. Ferhat Ozgur Catak

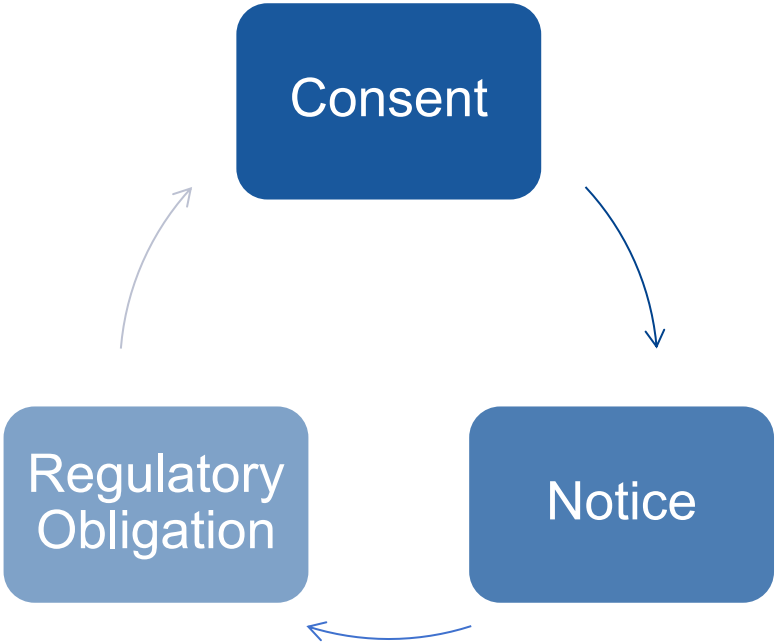
20.07.2020

Outline

- What is Data Privacy
- Perturbation Based Data Privacy
- Differential Privacy
- Homomorphic Encryption
- Order Preserving Encryption

What is Data Privacy

- It is a branch of data security concerned with the **proper handling of data**



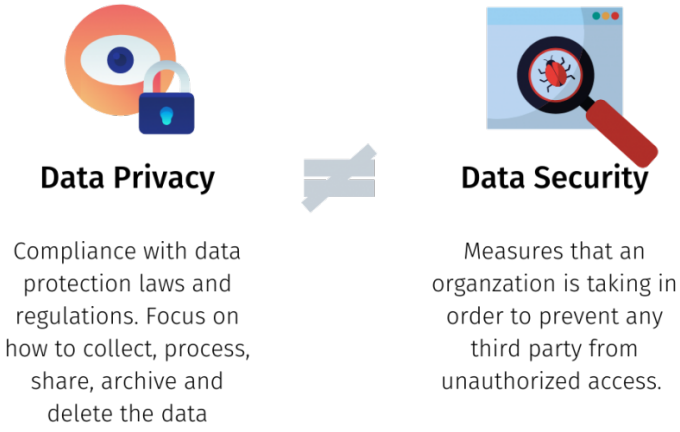
Data Privacy vs Data Security

Data Security definition

- **It** regards to the **means of protection** that an organization is taking in order **to prevent any third party from unauthorized access** to digital data.

Data Privacy definition

- It focuses on the **rights of individuals**, the purpose of data collection and processing, privacy preferences, and the way organizations govern personal data of data subjects.



Privacy-Enhancing Technologies (PETs)

- PETs allow online users to protect the privacy of their personally identifiable information (PII) provided to and handled by services or applications.
- PETs use techniques to **minimize possession** of personal data **without losing the functionality** of an information system

Families of PETs		
	Soft privacy technologies (The third-party can be trusted for the processing of data)	Access control
		Differential Privacy
		Tunnel encryption (SLL/TLS)
	Hard privacy technologies (Third-parties cannot be trusted.)	Homomorphic Encryption
		Secure multi-party computation
		Order preserving encryption

Privacy Threats in Data Analytics

Surveillance	• Based on the opinion data and sentiment analysis, social media sites does provide recommendations of the new friends, places to visit, people to follow etc. This is possible only when they continuously monitor their customer's transactions.
Disclosure	• Consider a hospital holding patient's data which include (Zip, gender, age, disease). The third party data analyst can map this information with the freely available external data sources like census data and can identify person suffering with some disorder.
Discrimination	• It can happen when some private information of a person is disclosed. For instance, statistical analysis of electoral results proved that people of one community were completely against the party, which formed the government.
Personal embracement and abuse	• For example, a person was privately undergoing medication for some specific problem and was buying some medicines on a regular basis from a medical shop. As part of their regular business model, the medical shop may send some reminder and offers related to these medicines over phone. If any family member has noticed this, it will lead to personal embracement and even abuse

Data Perturbation based Data Privacy

How do you publicly release a database without compromising individual privacy?

K-Anonimity

L-Diversity

T-closeness

K-Anonimity

- It was proposed by Samati and Sweeney in 1998.
- **Attacker model**: Creating a connection using quasi-identifier (identity disclosure)
- **Solution**: Creating indistinguishable data sets

Age	Gender	Zip Code
34	m	81667
45	f	81675
34	f	81931
45	m	81925
70	f	81931
70	m	81931
66	m	80931

2-anonimity

Age	Gender	Zip Code
< 50	*	816**
< 50	*	816**
< 50	*	819**
< 50	*	819**
≥ 50	*	819**
≥ 50	*	819**
*	*	*

Suppression

K-Anonimity

- **K-Anonimity Feature Types**

Type	Feature	Example	Action
Key	can identify a person directly	Name, SSN	Remove or hide
Quasi Identifier	Can be linked to external data to identify the person	Zip code City Birth date Gender	Generalize
Sensitive	Sensitive data about the person	Income, disease	The relationship with the person should be cut off.

K-Anonimity

- **Example dataset**

Key	Quasi-identifier			Sensitive
Name	**Birth**	**Gender**	**Zipcode**	**Disease**
Andre	1/21/76	Male	53715	Heart Disease
Beth	4/13/86	Female	53715	Hepatitis
Carol	2/28/76	Male	53703	Brochitis
Dan	1/21/76	Male	53703	Broken Arm
Ellen	4/13/86	Female	53706	Flu
Eric	2/28/76	Female	53706	Hang Nail

k-Anonimity – Let's code

- We want to k-anonymize (**k=5**, at least 5 rows in each block) the adult.csv file
- I will use pure Pandas library to manipulate the data

```
adult["origin"] = adult.origin.apply(lambda x: "US"
                                     if x == "United-States"
                                     else "Non-US")
g = adult.groupby(group_by)
g = g.filter(lambda x:len(x) < 5)
g.sort_values(group_by, inplace=True)

print(tabulate(g[sel_cols].head(10),
               headers='keys',
               tablefmt='psql'))
```

```
+-------+----------+----------+----------+
| age   | gender   | origin   | income   |
|-------+----------+----------+----------|
+-------+----------+----------+----------+
```

K-Anonimity

- k-Anonymization does not provide privacy
- Sensitive values do not vary in a class label
  - Attacker should have prior knowledge

Homogeneity Attack

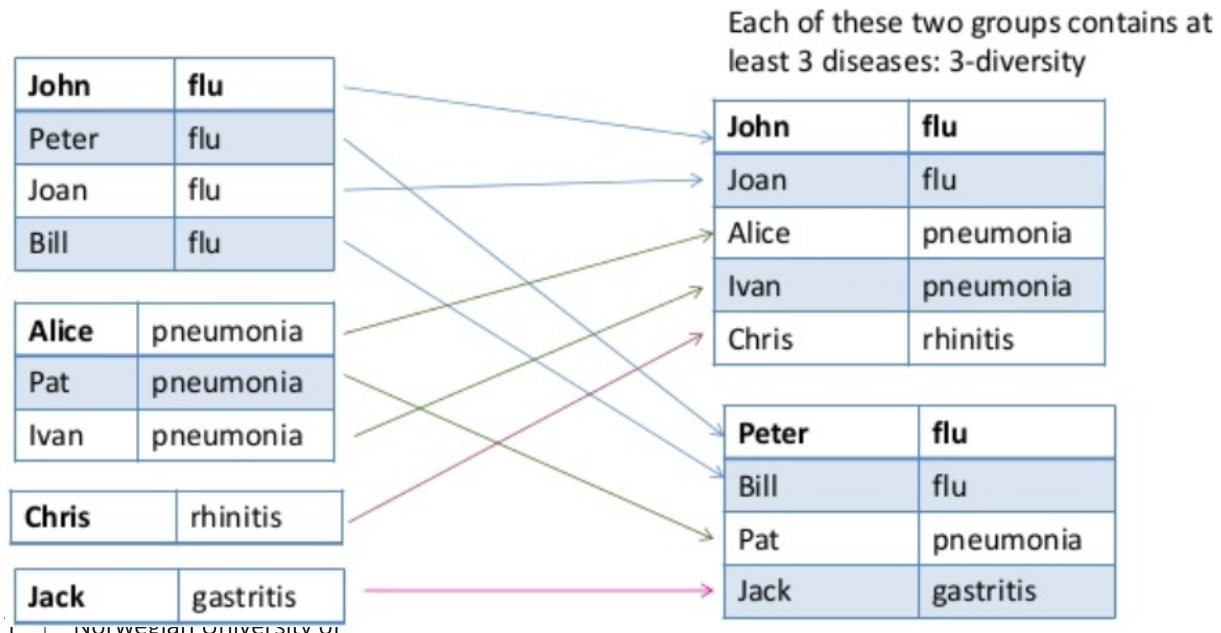
Bob	
Zipcode	***Age***
47678	27

A 3-anonymous patient table

Zipcode	Age	Disease
476**	2*	Heart Disease
476**	2*	Heart Disease
476**	2*	Heart Disease
4790*	≥40	Flu
4790*	≥40	Heart Disease
4790*	≥40	Cancer
476**	3*	Heart Disease
476**	3*	Cancer
476**	3*	Cancer

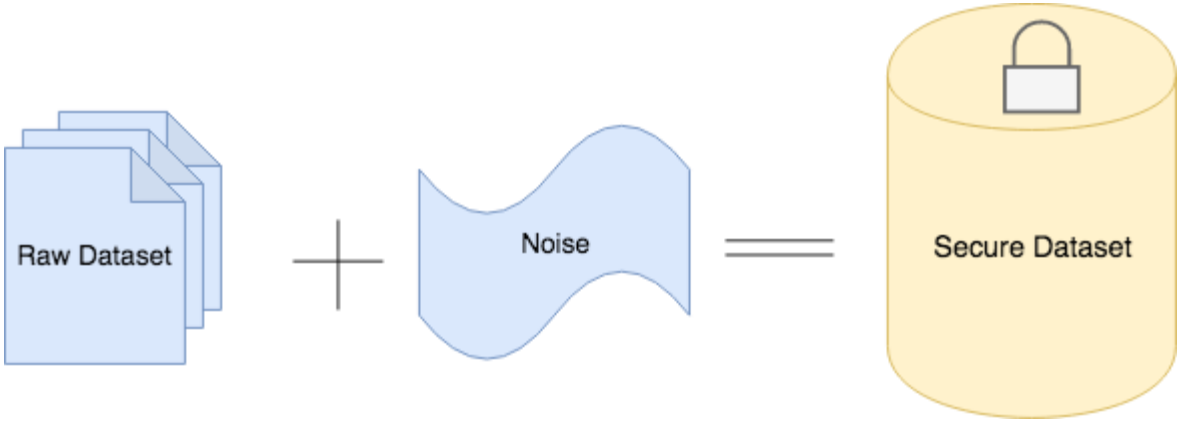
L-Diversity

- The k-Anonymity method is not concerned with the distribution of sensitive data. **Homogeneity attack**



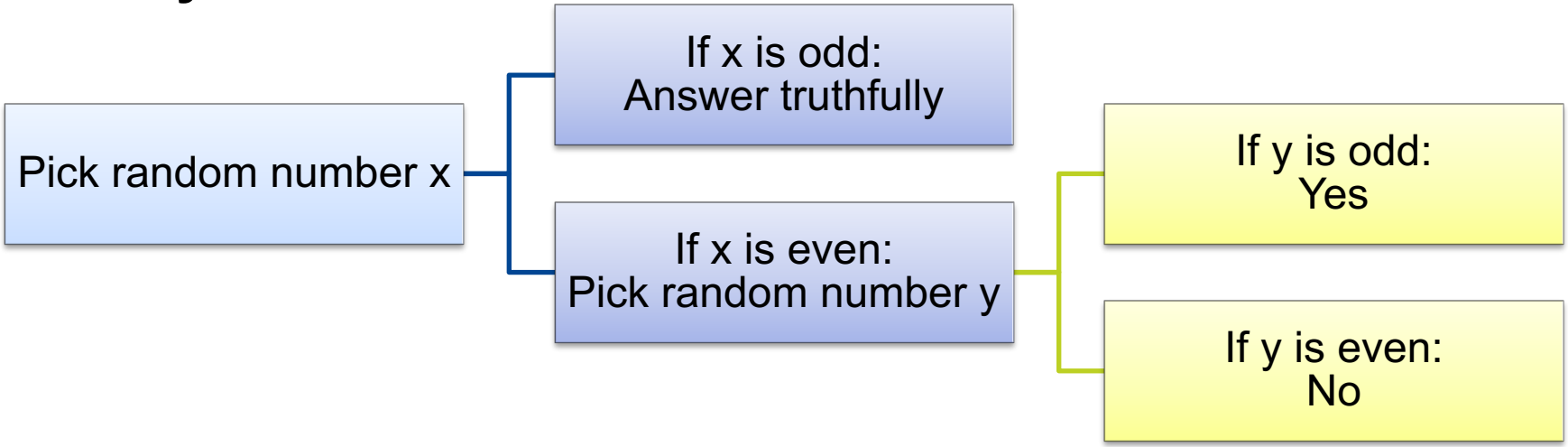
Differential Privacy (DP)

- DP protects user privacy by **adding random noise** to the data.
  - Adding three years to your age while answering a survey protects your information.



Differential Privacy (DP)

Have you ever break the law?



- Result of survey: 50% of time we get the Truth, 25% Yes, 25% No
- Every respondent has plausible deniability

What is Homomorphic Encryption?

- **Purpose**: Computation on encrypted data
  - data can remain **confidential** while it is processed in **untrusted environments**.
  - **Homomorphism**: is a structure-preserving map between two algebraic structures
    - describes the transformation of one data set into another while preserving relationships between elements in both sets.
  - Greek words for **"same structure."**
  - **Traditional Encryption vs Homomorphic Encryption**
    - **Homomorphic encryption** allows computation to be performed directly on **encrypted** data without requiring access to a secret key.
    - The result of such a computation remains in **encrypted** form, **and** can at a later point be revealed by the owner of the secret key.

What is Homomorphic Encryption?

No Trusted Third-Parties

- Data remains secure and private in untrusted environments
- The data stays encrypted at all times, which minimizes the likelihood that sensitive information ever gets compromised.

Tradeoff between data usability and privacy

- There is no need to mask or drop any features in order to preserve the privacy of data.
- All features may be used in an analysis, without compromising privacy.

Quantum-safe

- Fully homomorphic encryption schemes are resilient against quantum attacks

What is Homomorphic Encryption?

Limitations

Poor performance

- Between slow computation speed, fully homomorphic encryption remains problematic for computationally-heavy applications

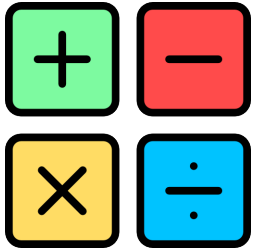
Large Memory Consumption

- Compared to plaintext operations making them sometimes impractical for the database queries

What is Homomorphic Encryption?

operation(plain)

Public key: (23,143)
Private key: (47,143)



Private What is the area? Width:7 height:3

Public

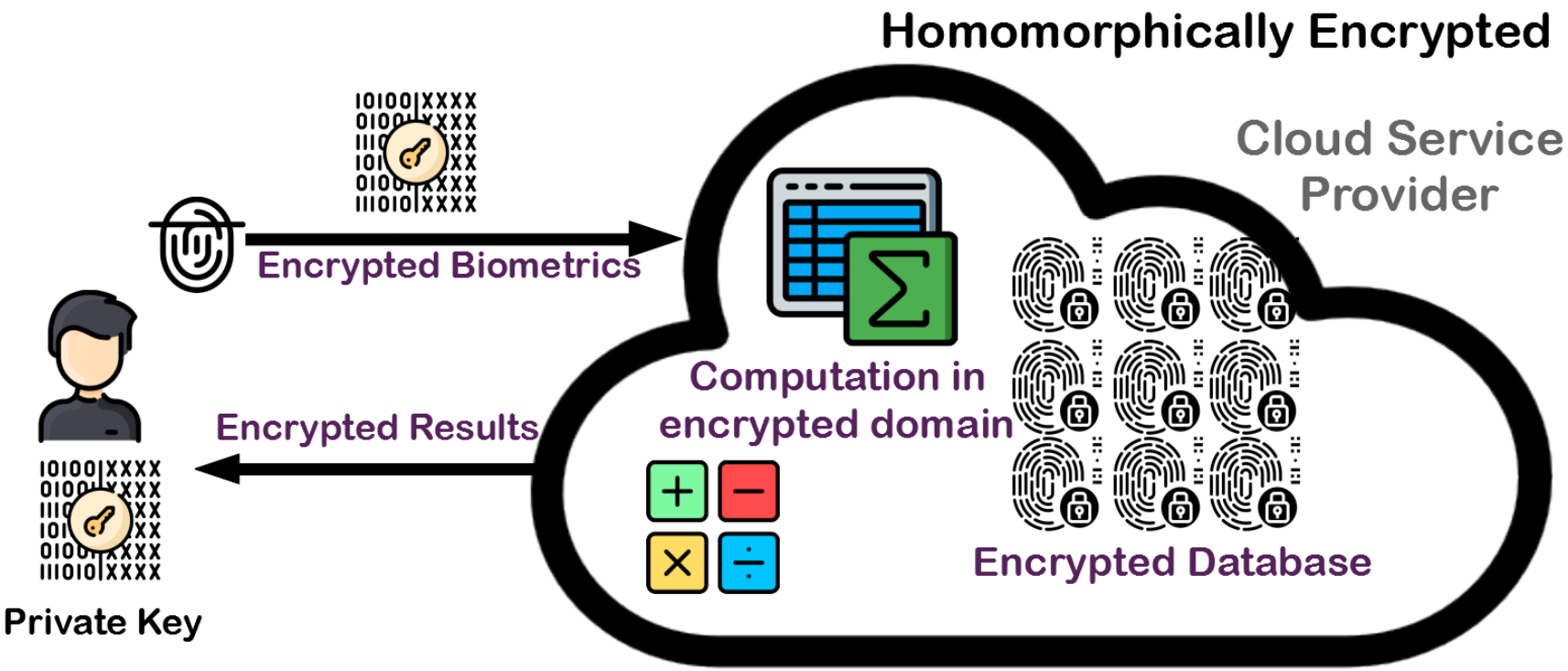
2 × 126 = 252

Enc(width) = width^{e} mod N
Enc(width) = 7^{23} mod 143
Enc(width) = 2

Enc(height) = height^{e} mod N
Enc(height) = 3^{23} mod 143
Enc(height) = 126

area = cipher^{d} mod N
area = 252^{47} mod 123
area = 21
7 × 3 = 21

What is Homomorphic Encryption?



Use-Cases

Medical Records

- Analyze disease/treatment without disclosing
- Search DNA markers without revealing DNA

SPAM Filtering

- Blacklisting Encrypted mails

Biometric Matching

- Encrypted distance metrics
- Similarity

Types of Homomorphic Encryption

- Partially Homomorphic
- Somewhat Homomorphic
- Fully Homomorphic

Partially Homomorphic Encryption

- When you can only perform certain mathematical operations on the ciphertext but not others
  - **RSA cryptosystem**: partially homomorphic with respect to **multiplication**
    - **[a] x [b] OK**
    - **[a] + [b] Not Ok**
  - **Caesar Cipher**: partially homomorphic with respect to **addition**
    - **[a] + [b] OK**
    - **[a] x [b] Not OK**
  - **Paillier:** partially homomorphic with respect to **addition**
    - **[a] + [b] OK**
    - **[a] x b Ok (Encrypted x Plain)**
    - **[a] x [b] Not OK**

Somewhat Homomorphic Encryption

- SHE is more general than PHE in the sense that it supports homomorphic operations with additions and multiplications. The drawback is that you can perform only a limited number homomorphic operations.

Fully Homomorphic Encryption

- When you can perform mathematical operations on the ciphertext
  - **BGV** and **BFV**
    - **(Encrypted with Encrypted)**
      - **[a] + [b] OK**
      - **[a] – [b] Ok**
      - **[a] x [b] OK**
    - **(Encrypted with Plain)**
      - **[a] + b OK**
      - **[a] – b Ok**
      - **[a] x b OK**

Open Source Libraries for HE

Microsoft

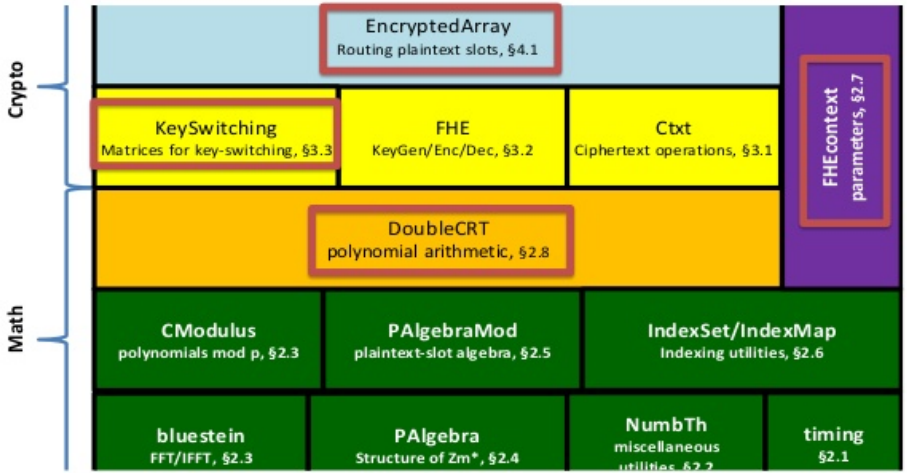
SEAL

Simple Encrypted Arithmetic Library

PyFHEL

- SEAL
- HeLib
- PALISADE

Architecture of HElib



Privacy-Preserving Area Solver

```python
from Pyfhel import Pyfhel

print("Step 1: Key generation")
HE = Pyfhel()
HE.contextGen(p=65537, m=2048, flagBatching=True)
HE.keyGen()

print("Step 2: Client encryption")
height = 3
width = 7
encrypted_height = HE.encryptInt(height)
encrypted_width = HE.encryptInt(width)

print("Step 3: Cloud computation")
encrypted_area = encrypted_height * encrypted_width

print("Step 4: Client decryption")
area = HE.decryptInt(encrypted_area)
print("Area:%d"%(area))
```

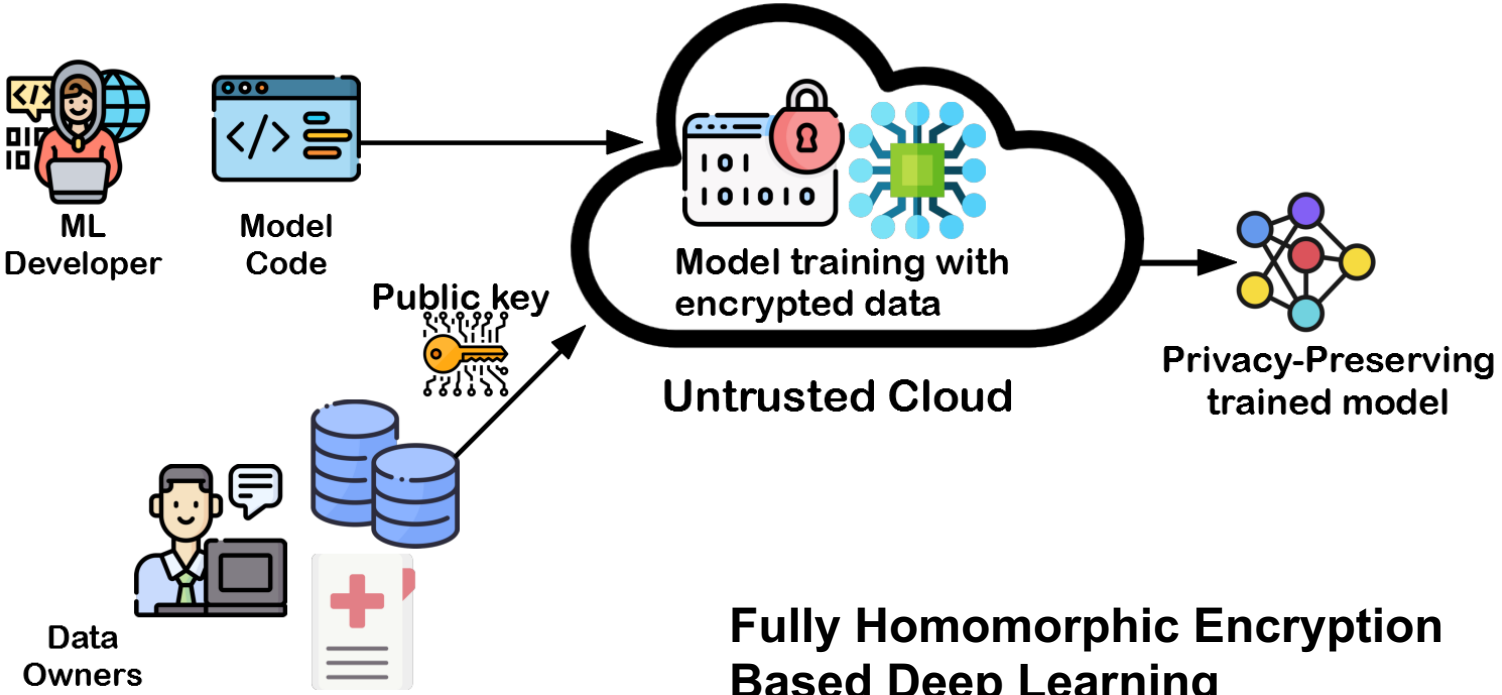
```
Step 1: Key generation
Step 2: Client encryption
Step 3: Cloud computation
Step 4: Client decryption
Area:21
```

Homomorphic Encryption Applications

- Privacy-Preserving Machine Learning

Homomorphic Encryption + **Secure Machine Learning** = **Privacy Preserving Machine Learning**

Homomorphic Encryption Applications



Fully Homomorphic Encryption Based Deep Learning

https://github.com/microsoft/CryptoNets

Order Preserving Encryption (OPE)

- A new concept with the cloud computing "Database as a Service (DBaaS)"
- *OPE is a encryption scheme whose encryption function preserves numerical ordering of the plaintexts.*
- It allows comparison operations to be directly applied on encrypted data, without decrypting the operands.
- SQL Complient applications
  - MAX, MIN, COUNT
  - GROUP BY, ORDER BY
  - SUM, AVG (Homomorphic encryption)

Value	Enc (Value)
1	0x0001102789d5f50b2beffd9f3dca4ea7
2	0x0065fda789ef4e272bcf102787a93903
3	0x009b5708e13665a7de14d3d824ca9f15
4	0x04e062ff507458f9be50497656ed654c
5	0x08db34fb1f807678d3f833c2194a759e

$$x < y \rightarrow Enc\,(x) < Enc\,(y)$$

OPE E

- T
- P
- T

```
### show the table
sql = "SELECT * FROM user_table order by salaries_enc"
curr = conn.execute(sql)
results = curr.fetchall()

x = PrettyTable(["Name", "Salary", "Encrypted salary"])

for r in results:
    x.add_row(r)
print(x)
```

Name	Salary	Encrypted salary
Christopher	1747	112303853
Brian	2322	150921129
Michael	2870	186655148
Lucas	3474	226011633
Ethan	3535	229697050
Alexis	4233	276437288
Craig	4496	292527718
Ryan	4687	305250029
Joshua	4761	310031780
Maria	4821	313982830

Contact Information

Dr. Ferhat Ozgur Catak
Norwegian University of Science and Technology
email: ferhat.o.catak@ntnu.no

Thank You!